

Algoritmos y Estructuras de Datos.

- Clase 6 - Algoritmos de Búsqueda y Ordenamiento -

Algoritmos de Búsqueda

- El problema de Búsqueda
- Cómo programar la Búsqueda Lineal "a mano"
- Búsqueda Lineal
- Buscar sobre una Lista Ordenada
 - Búsqueda Binaria

Algoritmos de Ordenamiento

- Ordenamiento por selección
 - Invariantes del Ordenamiento por Selección
 - Costo computacional del Ordenamiento por Selección
- Ordenamiento por inserción
 - Invariantes del Ordenamiento por Inserción
 - Costo computacional del Ordenamiento por Inserción
 - Inserción en una Lista Ordenada
- Algoritmos de Ordenamientos Recursivos
 - Ordenamiento por Mezcla 'Mergesort'
 - Costo computacional Mergesort?
 - Ordenamiento Rápido 'Quicksort'
 - Costo computacional Quicksort?
 - Una versión mejorada de Quick sort

Algoritmos de Búsqueda

- Búsqueda Lineal
- Buscar sobre una Lista Ordenada
 - Búsqueda Binaria

El problema de la **búsqueda** es uno de los problemas más clásicos de la computación, que se puede enunciar de la siguiente manera:

Dada una lista L y un valor x , devolver el índice de x en L si x está en L , y -1 (u otro valor representativo) si x no está en L .

Python nos provee la función `index()` para buscar en una Lista.

```
In [ ]: 1 # Ejemplo:
        2 [1,3,5,7].index(5)
```

```
In [ ]: 1 [1,3,5,7].index(20)
```

Como podemos ver, si el valor levanta una excepción. Queríamos un -1 u otro valor

Para solucionar este problema, Python también nos provee el operador `in`. Este operador nos permite averiguar si un elemento pertenece a una lista

```
In [ ]: 1 5 in [1,3,5,7] #[1,3,5,7].index(5)
```

```
In [ ]: 1 20 in [1, 3, 5, 7] # [1,3,5,7].index(20)
```

Utilizando ambos combinados, es decir, si llamamos a la función `index()` sólo cuando el resultado de `in` es verdadero, y devolvemos `-1` cuando el resultado de `in` es falso, estaremos resolviendo el problema planteado usando sólo funciones provistas por Python.

```
In [ ]: 1 # Ejemplo de Búsqueda Simple:
        2
        3 def busqueda_con_index(xs, x):
        4     """Busca un elemento x en una lista xs
        5         si x está en xs devuelve xs.index(x)
        6         de lo contrario devuelve -1 """
        7
        8     if x in xs:
        9         return(xs.index(x))
        10    else:
        11        return(-1)
        12
```

```
In [ ]: 1 busqueda_con_index([1, 4, 54, 3, 0, -1], 3)
```

```
In [ ]: 1 busqueda_con_index([1, 4, 54, 3, 0, -1], 44)
```

```
In [ ]: 1 busqueda_con_index([], 0)
```

¿Cuántas comparaciones hace este programa?

La pregunta es: ¿cuánto esfuerzo computacional requiere este programa?, es decir, ¿cuántas veces comparamos el valor que buscamos con los datos de la lista?

No lo sabemos porque no sabemos cómo están implementadas las funciones `in` e `index()`. La pregunta queda planteada por ahora pero daremos un método para averiguarlo más adelante en esta unidad (y de manera teorica formal en la próxima clase).

¿Cómo programar la búsqueda lineal "a mano"?

Cómo programamos una función sin las primitivas `in` e `index()`. Luego, esto nos permitirá estudiar una solución que puede portarse a otros lenguajes que no tienen instrucciones tan poderosas.

Solución (naïve):

Podemos comparar uno a uno los elementos de la lista con el valor de x , y retornar el valor de la posición donde lo encontramos en caso de encontrarlo. Si llegamos al final de la lista sin haber salido antes de la función es porque el valor de x no está en la lista, y en ese caso retornamos `-1`.

```
In [ ]: 1 # Ejemplo Búsqueda Lineal.
2
3 def busqueda_lineal(lista,x):
4     """ Búsqueda lineal.
5         Si x está en lista
6         contrario devuelve -1. """
7
8     # Estrategia: se recorren uno a uno los elementos de la lista
9     # y se los compara con el valor x buscado.
10
11     i=0 # i tiene la posición actual en la lista, comienza en 0
12     # el ciclo for recorre todos los elementos de lista:
13     for z in lista:
14         # estamos en la posición i, z contiene el valor de lista[i]
15
16         # si z es iguala x, devuelve i
17         if z==x:
18             return i
19
20         # si z es distinto de x, incrementa i, y continúa el ciclo
21         i+=1
22
23     # si salió del ciclo sin haber encontrado el valor, devuelve -1
24     return -1
```

```
In [ ]: 1 busqueda_lineal([1, 4, 54, 3, 0, -1], 44)
```

```
In [ ]: 1 busqueda_lineal([1, 4, 54, 3, 0, -1], 3)
```

```
In [ ]: 1 busqueda_lineal([1, 4, 54, 3, 0, -1], 0)
```

```
In [ ]: 1 busqueda_lineal([], 0)
```

¿Cuántas comparaciones hace este programa?

El código es un ciclo que recorre uno a uno los elementos de la lista, y en el cuerpo de ese ciclo, donde se compara cada elemento con el valor buscado. En el caso de encontrarlo se devuelve la posición. Si el valor no está en la lista se recorrerá la lista entera, haciendo una comparación por elemento (*peor caso*).

- Si el valor está en la posición p de la lista se hacen p comparaciones; y
- si el valor no está se hacen tantas comparaciones como elementos tenga la lista (n).

La cantidad de comparaciones para encontrar un valor arbitrario crecerá en forma proporcional al tamaño de la lista. Diremos que este algoritmo tiene un comportamiento proporcional a la longitud de la lista involucrada, o que es un algoritmo **lineal**, ya que en el *peor caso* es necesario realizar n comparaciones.

Buscar sobre una lista ordenada

Tener una Lista ordenada nos proporciona una estructura sobre la cual podemos facilitar la búsqueda de elementos.

Búsqueda binaria

Aprovecharemos el hecho de que la lista está ordenada y vamos a dividir nuestro espacio de búsqueda y se irá reduciendo a segmentos cada vez menores de la lista original. La idea es descartar segmentos de la lista donde el valor seguro que no puede estar.

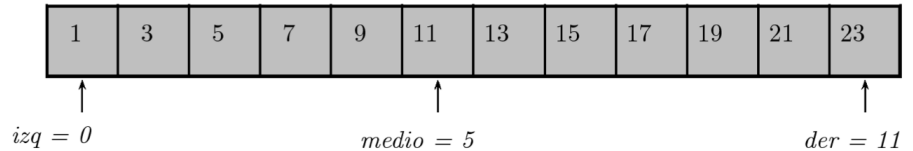
Pasos del Algoritmo:

1. Consideramos como segmento inicial de búsqueda a la lista completa.
2. Analizamos el punto medio del segmento (el valor central), si es el valor buscado, devolvemos el índice del punto medio.
3. Si el valor central es mayor al buscado, podemos descartar el segmento que está desde el punto medio hacia la derecha.

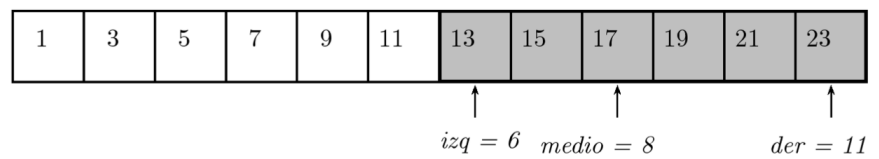
4. Si el valor central es menor al buscado, podemos descartar el segmento que está desde el punto medio hacia la izquierda.
5. Una vez descartado el segmento que no nos interesa, volvemos a analizar el segmento restante, de la misma forma.
6. Si en algún momento el segmento a analizar tiene longitud 0 o negativa significa que el valor buscado no se encuentra en la lista.

Usamos dos variables (*izq* y *der*) que contienen la posición de *inicio* y la posición de *fin* del segmento que se está usando. Usaremos la variable *medio* para contener la posición del *punto medio* del segmento.

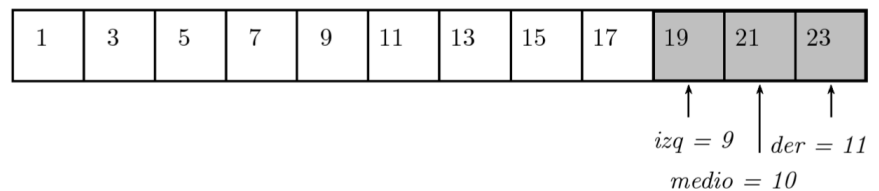
El arreglo inicial:



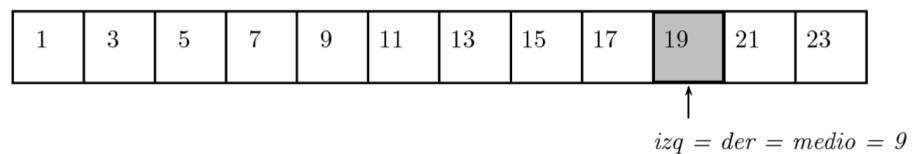
Paso 2 (lista[5]<18):



Paso 3 (lista[8]<18):



Paso 4 (lista[9]>=18):



Busqueda del valor 18 en la Lista [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23] .

Como no se encontró al valor buscado, devuelve -1.

```
In [ ]: 1 # Ejemplo Búsqueda Binaria
2
3 def busqueda_binaria(lista, x):
4     """Búsqueda binaria
5         Precondición: lista está ordenada
6         Devuelve -1 si x no está en lista;
7         Devuelve p tal que lista[p] == x, si x está en lista"""
8
9     # Busca en toda la lista dividiéndola en segmentos y considerando
10    # a la lista completa como el segmento que empieza en 0 y termina
11    # en len(lista) - 1.
12
13    izq = 0 # izq guarda el índice del comienzo del segmento
14    der = len(lista) - 1 # der guarda el índice del fin del segmento
15
16    # un segmento es vacío cuando izq
17    while izq <= der:
18        # el punto medio del segmento
19        medio = int((izq+der)/2)
20
21        print("DEBUG:", "izq:", izq, "der:", der, "medio:", medio)
22        # si el medio es igual al valor buscado, lo devuelve
23        if lista[medio] == x:
24            return medio
25        # si el valor del punto medio es mayor que x, sigue buscando
26        # en el segmento de la izquierda: [izq, medio-1], descartando la
27        # derecha
28        elif lista[medio] > x:
29            der = medio-1
30        # sino, sigue buscando en el segmento de la derecha:
31        # [medio+1, der], descartando la izquierda
32        else:
33            izq = medio+1
34        # si no salió del ciclo, vuelve a iterar con el nuevo segmento
35
36    # salió del ciclo de manera no exitosa: el valor no fue encontrado
37    return -1
38
```

```
In [ ]: 1 lista = [0, 1, 3, 4, 54, 68] #input ("Dame una lista ordenada ([[ ]] para terminar): ")
2 while lista != [[ ]]:
3     x = 1 #input("¿Valor buscado?: ")
4     resultado = busqueda_binaria(lista, x)
5     print("Resultado: ", resultado)
6     #lista = input ("Dame una lista ordenada ([[ ]] para terminar): ")
```

```
In [ ]: 1 lista = [0, 1, 3, 4, 54, 68]
2 resultado = busqueda_binaria(lista, 1)
```

¿Cuántas comparaciones hace este programa?

Para responder esto **siempre pensaremos en el peor caso**, es decir, que se descartaron varias veces partes del segmento para finalmente llegar a un segmento vacío, **cundo el valor buscado no se encontraba en la lista**.

En cada paso el segmento se divide por la mitad, y en cada paso se hace una comparación con el valor buscado. Por lo tanto, la cantidad de comparaciones que hacen con el valor buscado es aproximadamente igual a la cantidad de pasos necesarios para llegar a un segmento de tamaño 1.

Sea N la longitud de la lista, es más sencillo para abordar, suponer que N (la longitud) es potencia de 2, es decir $\text{len}(lista) = 2^k = N$

- Luego del primer paso, el segmento a tratar es de tamaño $N = \frac{N}{2^0}$, equiv. esto es 2^k .
- Luego del segundo paso, el segmento a tratar es de tamaño $\frac{N}{2} = \frac{N}{2^1}$, equiv. esto es 2^{k-1} .
- Luego del tercer paso, el segmento a tratar es de tamaño $\frac{N}{4} = \frac{N}{2^2}$, equiv. esto es 2^{k-2} .
- ...
- Luego del paso k , el segmento a tratar es de tamaño $\frac{N}{N} = \frac{N}{2^k} = 1$, equiv. esto es $2^{k-k} = 1$.

Este programa hace aproximadamente k comparaciones con el valor buscado cuando $\text{len}(\text{lista}) = N = 2^k$.
Despejando k de la ecuación anterior:

$$T(N) = \frac{N}{2^k} \quad \text{cuando} \quad \frac{N}{2^k} = 1$$

$$\Rightarrow N = 2^k * 1$$

$$\Rightarrow \log_2(N) = k * \underbrace{\log_2(2)}_{\log_2(2)=1}$$

$$\Rightarrow k = \log_2(N)$$

Podemos ver que este programa realiza aproximadamente $\log_2(\text{len}(\text{lista})) = \log_2(N)$ comparaciones.

Cuando usamos una potencia distinta de 2 el razonamiento es menos prolijo, pero $\log_2(N)$ es una cota apropiada como referencia.

Algoritmos de Ordenamiento

- Ordenamiento por selección
- Ordenamiento por inserción

Las Listas de Python, poseen un método `sort` que las ordena de menor a mayor. No todas las estructuras cuentan con un método `sort` que las ordene. Vamos a ver cómo se hace para ordenar cuando no hay un método `sort`, y cuánto cuesta ordenar.

Tenemos varias maneras de ordenar, y no todas tienen mismo costo computacional. Vamos a ir viendo las más sencillas de escribir, que en general suelen ser aquellas con mayor costo computacional.

Ordenamiento por Selección

El método de ordenamiento se basa en la siguiente idea:

Pasos del Algoritmo:

- 1: Buscar el mayor de todos los elementos de la lista.

3	2	-1	5	0	2
---	---	----	---	---	---

Encuentra el valor 5 en la posición 3.

- 2: Poner el mayor al final (intercambiar el que está en la última posición de la lista con el mayor encontrado).

3	2	-1	2	0	5
---	---	----	---	---	---

Intercambia el elemento de la posición 3 con el de la posición 5.
En la última posición de la lista está el mayor de todos.

- 3: Buscar el mayor de todos los elementos del segmento de la lista entre la primera y la anteúltima posición.

3	2	-1	2	0	5
---	---	----	---	---	---

Encuentra el valor 3 en la posición 0.

- 4: Poner el mayor al final del segmento (intercambiar el que está en la última posición del segmento—o sea anteúltima posición de la lista—con el mayor encontrado).

-1	0	2	2	3	5
----	---	---	---	---	---

La lista se encuentra ordenada.

• ...

- Paso n: Se termina cuando queda un único elemento sin tratar: el que está en la primera posición de la lista, y que es el menor de todos porque todos los mayores fueron reubicados.

3	2	-1	5	0	2
---	---	----	---	---	---

Encuentra el valor 5 en la posición 3.

```
In [ ]: 1 def ord_seleccion(lista):
2         """Ordena una lista de elementos según el método de selección.
3           Pre: los elementos de la lista deben ser comparables.
4           Post: la lista está ordenada. """
5
6         for i in range(len(lista)):
7             for j in range(i+1, len(lista)):
8                 if lista[i] > lista[j]:
9                     # Intercambiar
10                    lista[i], lista[j] = lista[j], lista[i]
11
```

```
In [ ]: 1 l=[3, 2, -1, 5, 0, 2]
2 ord_seleccion(l)
3 print(l)
```

```
In [ ]: 1 l=[]
2 ord_seleccion(l)
3 print(l)
4
5 l=[1]
6 ord_seleccion(l)
7 print(l)
```

```
In [ ]: 1 l=[1,2,3,4,5]
2 ord_seleccion(l)
```

Invariante en el Ordenamiento por Selección

Todo ordenamiento tiene un **invariante** que permite asegurarse de que cada paso que se toma va en la dirección de obtener una lista ordenada.

La invariante para el caso del ordenamiento por selección: "los elementos desde $n + 1$ hasta el final de la lista están ordenados y son mayores que los elementos de 0 a n , es decir que ya están en su posición definitiva".

¿Cuántas comparaciones/intercambios hace este programa?

En el código de la función tiene dos ciclos anidados, el primero realizara $\text{len}(lista) = N$ iteraciones. Luego, recorreremos el segmento de la lista desde i hasta N , para buscar el máximo elemento.

Por lo que en nuestro caso debemos recorrer en el primer paso N elementos, en el segundo paso $N - 1$ elementos, en el tercer paso $N - 2$ elementos, etc. Cada uso de un elemento implica una cantidad constante y pequeña de comparaciones (que no es dependiente de N).

$$\begin{aligned} T(N) &= N + (N - 1) + (N - 2) + \dots + 2 + 1 \\ &= \sum_{i=1}^N i = \frac{n(n+1)}{2} \\ &= \frac{N^2 + N}{2} \\ &= N^2 * \frac{1}{2} + N * \frac{1}{2} \end{aligned}$$

Como podemos ver la cantidad de comparaciones que el algoritmo realiza es del orden de N^2 .

$$T(N) \approx c * (2 + 3 + \dots + N) \approx c * \frac{N * (N + 1)}{2} \sim N^2$$

Para Ordenar por Selección una Lista de tamaño N el tiempo del es del orden de N^2 .

Cabe destacar, que este tiempo es independiente de si la lista estaba previamente ordenada o no.

Ver Notebook **Clase6-Ejemplos** para el cálculo "paso a paso".

Ordenamiento por Inserción

Ordenamiento por inserción

El método de ordenamiento se basa en la siguiente idea:

Pasos del Algoritmo:

- 0: Partimos de la misma lista de ejemplo utilizada para el ordenamiento por selección.

3	2	-1	5	0	2
---	---	----	---	---	---

- 1: Considerar el segundo elemento de la lista, y ordenarlo respecto del primero, desplazándolo hasta la posición correcta, si corresponde.

2	3	-1	5	0	2
---	---	----	---	---	---

Se desplaza el valor 2 antes de 3.

- 2: Considerar el tercer elemento de la lista, y ordenarlo respecto del primero y el segundo, desplazándolo hasta la posición correcta, si corresponde.

-1	2	3	5	0	2
----	---	---	---	---	---

Se desplaza el valor -1 antes de 2 y de 3.

- 3: Considerar el cuarto elemento de la lista, y ordenarlo respecto del primero, el segundo y el tercero, desplazándolo hasta la posición correcta, si corresponde.

-1	2	3	5	0	2
----	---	---	---	---	---

El 5 está correctamente ubicado respecto de $-1, 2$ y 3 (como el segmento hasta la tercera posición está ordenado, basta con comparar con el tercer elemento del segmento para verificarlo).

- ...
- $N-1$: Todos los elementos excepto el ante-último ya se encuentran ordenados.

-1	0	2	3	5	2
----	---	---	---	---	---

Todos los elementos excepto el ante-último ya se encuentran ordenados.

- N : Considerar el N -ésimo elemento de la lista, y ordenarlo respecto del segmento formado por el primero hasta el $N - 1$ -ésimo, desplazándolo hasta la posición correcta, si corresponde.

-1	0	2	2	3	5
----	---	---	---	---	---

Se desplaza el valor 2 antes de 3 y de 5.

```

In [ ]: 1 def ord_insercion(lista):
2         """ Ordena una lista de elementos según el método de inserción.
3             Pre: los elementos de la lista deben ser comparables.
4             Post: la lista está ordenada. """
5
6         # i va desde la primera hasta la penúltima posición de la lista
7         for i in range(len(lista)-1):
8             # Si el elemento de la posición i+1 está desordenado respecto
9             # al de la posición i, reubicarlo dentro del segmento [0:i]
10            if lista[i+1] < lista[i]:
11                reubicar(lista, i+1)
12
13            print("DEBUG: ", lista)
14
15 def reubicar(lista, p):
16     """ Reubica al elemento que está en la posición p de la lista
17         dentro del segmento [0:p-1].
18         Pre: p tiene que ser una posición válida de lista. """
19
20     #v es el valor a reubicar
21     v = lista[p]
22
23     # Recorre el segmento [0:p-1] de derecha a izquierda hasta
24     # encontrar
25     j = p
26     while j > 0 and v < lista[j-1]:
27         # Desplaza los elementos hacia la derecha, dejando lugar
28         # para insertar el elemento v donde corresponda.
29         lista[j] = lista[j-1]
30         # Se mueve un lugar a la izquierda
31         j -= 1
32
33     # Ubica el valor v en su nueva posición
34     lista[j] = v
35

```

```

In [ ]: 1 l=[3, 2,-1,5, 0, 2]
2 ord_insercion(l)
3 print(l)

```

```

In [ ]: 1 l=[]
2 ord_insercion(l)
3 print(l)
4
5 l=[1]
6 ord_insercion(l)
7 print(l)

```

```

In [ ]: 1 l=[1,2,3,4,5,6]
2 ord_insercion(l)
3 print(l)

```

Invariante del Ordenamiento por Inserción

En el Ordenamiento por Inserción, a cada paso se considera que los elementos que se encuentran en el segmento de 0 a i están ordenados, de manera que agregar un nuevo elemento implica colocarlo en la posición correspondiente y el segmento seguirá ordenado.

¿Cuántas inserciones hace este programa?

La función principal avanza por la lista de izquierda a derecha, mientras que la función `reubicar` cambia los elementos de lugar de derecha a izquierda.

El peor caso es cuando un elemento que está en la posición j es que deba ser ubicado al principio de la lista. En general, el peor para ordenar la lista es que todos sus elementos deban ser reubicados.

Por ejemplo, en la lista `[10, 8, 6, 2, -2, -5]`, todos los elementos deben ser reubicados al principio de la lista.

Analizemos el algoritmo: en el *primer paso*, el segundo elemento se debe intercambiar con el primero; en el *segundo paso*, el tercer elemento se compara con el segundo y el primer elemento, y se ubica adelante de todo; en el *tercer paso*, el cuarto elemento se compara con el tercero, el segundo y el primer elemento, y se ubica adelante de todo; y así sucesivamente...

$$T(N) \approx c * (2 + 3 + \dots + N) \approx c * N * (N + 1)/2 \sim N^2$$

Al igual que el Ordenamiento por Selección, realizamos el mismo numero de comparaciones e inserciones. Es decir, ordenar por inserción una lista de tamaño N puede insumir (en el peor caso) tiempo del orden de N^2 .

Inserción en una Lista Ordenada

Es interesante a notar es que cuando la lista ya se encuentra ordenada, este algoritmo no hace ningún movimiento de elementos (intercambios), simplemente compara cada elemento con el anterior, y si es mayor sigue adelante.

Para el caso de una lista de N elementos que se encuentra ordenada, el tiempo que insume el algoritmo de inserción es:

$$T(N) \sim N$$

Es decir recorrer la lista una vez de principio a fin.

Algoritmos de Ordenamiento Recursivos

- Ordenamiento por Mezcla '*Merge sort*'
- Ordenamiento Rápido '*Quick sort*'
 - Una versión mejorada de *Quick sort*

Los métodos de ordenamiento que vimos hasta ahora son iterativos y cuyo tiempo estaba relacionado con el orden de N^2 .

Ahora veremos dos métodos de ordenamiento, basados éstos en un planteo recursivo del problema, los cuales nos permitirán ordenar de forma más eficiente (rápida).

Estos algoritmos usaran el método **Divide y Conquistaras** (*Divide and Conquer*) para resolver estos problemas. Ya hemos visto este comportamiento en el algoritmo de 'Busqueda Binaria'. En general podremos aplicar este método en situaciones donde vale siguiente principio:

- Analizar el problema y **dividirlo en varios 'sub-problemas'** de tamaño menor.
- Luego **resolver cada uno de esos sub-problemas** por separado aplicando la misma técnica (en nuestro caso hemos dividido en sub-listas).
- Finalmente, **combinar las soluciones** parciales en una solución única

Como siempre, con las *soluciones recursivas*, debemos encontrar un **caso base** en el cual no se aplica la llamada recursiva (en nuestro caso el 'caso base' es: la '*lista es vacía o de tamaño 1*'). Además, **debemos asegurar que siempre se alcanzara el caso base**.

Ordenamiento por Mezcla. Merge sort

El método de ordenamiento se basa en la siguiente idea.

Pasos del Algoritmo:

1. **Caso Base:** si la lista es vacía o de tamaño 1, ya está ordenada y no hay nada que hacer.
2. Dividir la lista al medio, formando dos '*sub-listas*' de (aproximadamente) el mismo tamaño.
3. Ordenar cada una de esas dos sub-listas (llamada recursiva al método).
4. Una vez que ordenadas ambas sub-listas, intercalarlas de manera ordenada.

Ejemplo:

Usaremos en método *Mergesort* para ordenar la lista [6, 7, -1, 0, 5, 2, 3, 8] .

Deberemo sordenar recursivamente las sub-listas `[6, 7, -1, 0]` y `[5, 2, 3, 8]` . Obtendremos las sub-listas ordenadas `[-1, 0, 6, 7]` y `[2, 3, 5, 8]` . Si intercalamos ordenadamente las dos sub-listas ordenadas, obtenemos la solución: `[-1, 0, 2, 3, 5, 6, 7, 8]` .

Función `merge_sort(lista)` :

1. **Caso Base:** lista es vacía o de tamaño 1. Es decir que ya está ordenada, nada para hacer. Devolvemos la lista tal cual.
2. De lo contrario:
 - 2.1 `medio = len(lista)/2`
 - 2.2 `izq = merge_sort(lista[:m])`
 - 2.3 `der = merge_sort(lista[m:])`
 - 2.4 Retornamos `merge(izq, der)`

Ahora falta la función `merge` que realiza la intercalación ordenada de dos listas (ordenadas). '*Dadas dos listas ordenadas se debe obtener una nueva lista que resulte de intercalar a ambas de manera ordenada*'.

Función `merge(lista1, lista2)` :

1. Utilizaremos dos índices, `i` y `j` , para recorrer cada una de las dos listas.
2. Utilizaremos una tercera lista, `resultado` , donde almacenaremos el resultado.
3. Mientras `i` sea menor que el largo de `lista1` y `j` menor que el largo de `lista2` , tenemos elementos para comparar en ambas listas.
 - 3.1 El menor es el de `lista1` :
 - 3.1.1 Agregar el elemento `i` de `lista1` al final del resultado.
 - 3.1.2 Avanzar el índice `i` .
 - 3.2 El menor es el de `lista2` :
 - 3.2.1 Agregar el elemento `j` de `lista2` al final del resultado.
 - 3.2.2 Avanzar el índice `j` .
4. Cuando una de las dos listas se termina, simplemente hay que agregar todo lo que queda en la otra al final del resultado.

```

In [ ]: 1  ## MergeSort
        2
        3  def merge_sort(lista):
        4      """ Ordena lista mediante el método merge sort.
        5          Pre: lista debe contener elementos comparables.
        6          Devuelve: una nueva lista ordenada. """
        7
        8      # Caso Base: la lista contiene de 1 o 0 elementos, **ya está ordenada**.
        9      if len(lista) < 2:
       10          return lista
       11
       12      # La lista tiene 2 o más elementos, dividimos y ordenamos cada parte.
       13      medio = len(lista) // 2
       14      izq = merge_sort(lista[:medio])
       15      der = merge_sort(lista[medio:])
       16      return merge(izq, der)
       17
       18
       19  def merge(listal, lista2):
       20      """ Intercala los elementos de listal y lista2 de forma ordenada.
       21          Pre: listal y lista2 deben estar ordenadas.
       22          Devuelve: una lista con los elementos de listal y lista2. """
       23
       24      i, j = 0, 0
       25      resultado = []
       26
       27      # Intercalar ordenadamente
       28      while(i < len(listal) and j < len(lista2)):
       29          if (listal[i] < lista2[j]):
       30              resultado.append(listal[i])
       31              i += 1
       32          else:
       33              resultado.append(lista2[j])
       34              j += 1
       35
       36      # Si una lista se ha terminado, agregamos lo que falta,
       37      # es decir si i o j ya son len(lista), entonces no agregaremos nada.
       38      resultado += listal[i:]
       39      resultado += lista2[j:]
       40      return resultado
       41

```

```

In [ ]: 1  ## Ejemplo de Uso.
        2
        3  l=[6, 7, -1, 0, 5, 2, 3, 8]
        4  merge_sort(l)
        5  print(l)
        6

```

¿Cuánto cuesta el MergeSort?

Sea N la longitud de la lista.

- Para intercalar dos listas de longitud $N/2$ hace falta recorrer ambas listas, N elementos (es proporcional a N). Llamemos $a * N$ a ese tiempo.
- Llamamos $T(N)$ al tiempo que tarda el algoritmo en ordenar una lista de longitud N . Podemos ver que $T(N) = 2 * T(N/2) + a * N$.
- Cuando la lista es vacía o de tamaño 1, la operación es de tiempo constante: $T(1) = T(0) = b$.

Nuevamente simplificaremos la cuenta suponiendo que $N = 2^k$.

$$\begin{aligned}
T(N) = T(2^k) &= 2 * T(2^{k-1}) + a * 2^k \\
&= 2 * (2 * T(2^{k-2}) + a * 2^{k-1}) + a * 2^k \\
&= 2^2 * T(2^{k-2}) + a * 2^k + a * 2^k \\
&= \vdots \\
&= 2^i * T(2^{k-i}) + i * a * 2^k \\
&= \vdots \\
&= 2^k * T(1) + k * a * 2^k \\
&= b * 2^k + k * a * 2^k
\end{aligned}$$

Si $N = 2^k$ entonces $k = \log_2(N)$, y por lo tanto hemos demostrado que:

$$T(N) = bN + aN \log_2(N) \equiv N * \log_2(N)$$

Nos interesa es aproximar este valor de forma más general, removiendo los términos de menor orden, obtendremos que $T(N) \sim N * \log_2 N$. Este algoritmo que se *porta mucho mejor* que los anteriores.

Ordenamiento por Rápido. Quick sort

QuickSort el algoritmo de ordenamiento más famoso. Esto radica en que en la práctica, con casos reales, es uno de los algoritmos más eficientes para ordenar.

El método se basa en la siguiente idea.

Pasos del Algoritmo:

1. **Caso Base:** la lista es vacía o de tamaño 1, ya está ordenada y no hay nada que hacer.
2. Tomar un elemento de la lista (por ejemplo el primero) al que llamaremos **pivote** y armar tres sub-listas:
 - 2.1 la de todos los elementos de la lista menores al pivote,
 - 2.2 la formada sólo por el pivote, y
 - 2.3 la de los elementos mayores o iguales al pivote, pero sin contarlos al pivote.
3. Ordenar las tres sub-listas (llamada recursiva).
4. Recolectar los resultados, concatenar las tres sub-listas (ya ordenadas).

Ejemplo:

La lista original es $[6, 7, -1, 0, 5, 2, 3, 8]$ consideramos que el **pivote** es el primer elemento (6) y armamos las sub-listas $[-1, 0, 5, 2, 3]$, $[6]$ y $[7, 8]$. Las ordenamos recursivamente $[-1, 0, 5, 2, 3]$ (obtenemos $[-1, 0, 2, 3, 5]$) y $[7, 8]$ (obtenemos la misma) y concatenamos en el orden adecuado, y así obtenemos $[-1, 0, 2, 3, 5, 6, 7, 8]$.

La parte más importante es conseguir armar las tres sub-listas en las que se parte la lista original. Definimos la función auxiliar `_partition` que recibe una lista no vacía y devuelve las tres sub-listas `menores`, `medio` y `mayores` (incluye los iguales, de haberlos) en las que se parte la lista original usando como pivote al primer elemento.

Diseño del QuickSort:

1. **Caso Base:** la lista es vacía o de tamaño 1. Es decir que ya está ordenada y no hay nada que hacer. Devolvemos lista tal cual.
2. De lo contrario:
 - 2.1 Dividir la lista en tres, usando `_partition`.
 - 2.2 Llamada recursiva a `quick_sort(menores)`, `quick_sort(mayores)`, y concatenarlo con `medio` en el medio.

Función `_partition`:

0. **Precondición!!** "la lista es no vacía".
1. Se elige el primer elemento como pivote.

2. Se inicializan como vacías las listas menores y mayores .
3. Para cada elemento de la lista después del primero:
 - 3.1 Si es menor que el pivote, se lo agrega a menores.
 - 3.2 De lo contrario, se lo a agrega a mayores.
4. Devolver menores, [pivote] , mayores

```
In [ ]: 1 def quick_sort(lista):
2         """ Ordena la lista de forma recursiva.
3             Pre: los elementos de la lista deben ser comparables.
4             Devuelve: una nueva lista con los elementos ordenados. """
5
6         # Caso base
7         if len(lista) < 2:
8             return lista
9         # Caso recursivo
10        menores, medio, mayores = _partition(lista)
11        return quick_sort(menores) + medio + quick_sort(mayores)
12
13
14 def _partition(lista):
15     """ Pre: lista no vacía.
16         Devuelve: tres listas: menores, medio y mayores. """
17
18     pivote = lista[0]
19     menores , mayores = [], []
20
21     for x in range(1, len(lista)):
22         if lista[x] < pivote:
23             menores.append(lista[x])
24         else:
25             mayores.append(lista[x])
26
27     return menores, [pivote], mayores
```

```
In [ ]: 1
```

¿Cuánto cuesta el QuickSort?

La ecuación del tiempo consumido parece ser la misma que en el MergeSort: Una partición que se hace en tiempo lineal más dos llamadas recursivas a mitades de la lista original.

Aquí la partición, tomando como pivote `lista[0]` no siempre parte la lista en mitades iguales, puede suceder--en el peor caso--que se parta la lista en `([], [lista[0]], lista[1:])` (esto es lo que pasa cuando la lista estaba ya ordenada), y en ese caso se comporta como selección.

Cuando la lista tiene números ubicados de manera arbitraria, podemos estimar un comportamiento parecido al MergeSort, y por lo tanto:

$$T(N) \sim N * \log_2(N)$$

Una versión mejorada de QuickSort

Podemos mejorar el desempeño del algoritmo Quick Sort. Para ello debemos optimizar las particiones de manera mas eficiente, operando sobre la misma lista recibida, y reubicando los elementos en su interior, de modo que no se consuma el doble de memoria.

En este caso, tendremos una función `_quick_sort` , analoga a la vista anteriormente, con la particularidad de que en lugar de recibir listas cada vez más pequeñas, solo trabajaremos con los índices de *inicio* y *fin* que indican la porción de la lista sobre la que debe operar.

Escribiremos una función `quick_sort` , que recibirá la lista y parámetros, y se encargará de llamar a la función `_quick_sort` con los índices correspondientes.

La función `_partition` también recibirá los índices de *inicio* y *fin*. En este caso, la función se encargará de cambiar de lugar los elementos de la lista, de modo que todos los menores al pivote se encuentren antes de él y todos los mayores se encuentren después.

Pasos de la función `_partition`:

1. Elegir el pivote como el primero de los elementos a procesar.
2. Inicializar un índice `menores` con el valor del primer elemento de la porción a procesar.
3. Recorrer los elementos desde el segundo hasta el último a procesar:
 - 3.1 Si el elemento es menor al pivote, incrementar el índice `menores` y de ser necesario, intercambiar el elemento para que pase a ser el último de los menores.
4. Intercambiar el pivote con el último de los menores
5. Devolver la posición del pivote.

Este código, si bien más complejo, cumple con el objetivo de proveer un algoritmo de ordenamiento que en el caso promedio tarda $T(N) \sim N * \log_2(N)$.

```
In [ ]: 1 def quick_sort(lista):
2         """ Ordena la lista de forma recursiva.
3             Pre: los elementos de la lista deben ser comparables.
4             Post: la lista está ordenada. """
5
6         quick_sort(lista, 0, len(lista) - 1)
7
8     def _quick_sort(lista, inicio, fin):
9         """ Función quick_sort recursiva.
10            Pre: los índices inicio y fin indican sobre qué porción operar.
11            Post: la lista está ordenada. """
12
13        # Caso base
14        if inicio >= fin:
15            return
16
17        # Caso recursivo
18        menores = _partition(lista, inicio, fin)
19        _quick_sort(lista, inicio, menores - 1)
20        _quick_sort(lista, menores + 1, fin)
21
22    def _partition(lista, inicio, fin):
23        """ Función partición que trabaja sobre la misma lista.
24            Pre: los índices inicio y fin indican sobre qué porción operar.
25            Post: los menores están antes que el pivote, los mayores después.
26            Devuelve: la posición en la que quedó ubicado el pivote. """
27
28        pivote = lista[inicio]
29        menores = inicio
30
31        # Cambia de lugar los elementos
32        for i in range(inicio+1, fin + 1):
33            if lista[i] < pivote:
34                menores += 1
35                if i != menores:
36                    _swap(lista, i, menores)
37
38        # Pone el pivote al final de los menores
39        if inicio != menores:
40            _swap(lista, inicio, menores)
41
42        # Devuelve la posición del pivote
43        return menores
44
45    def _swap(lista, i, j):
46        """ Intercambia los elementos i y j de lista. """
47        lista[j], lista[i] = lista[i], lista[j]
48
```

Podemos seguir mejorando el algoritmo QuickSort?

La elección es sumamente importante. Distintos análisis han confirmado que algunas elecciones de pivote son mas efectivas. Cuando realizamos nuestra implementación del algoritmo, no usaremos cualquier pivote. Dpendiendo del problema a ordenar, haremos una "decisión educada". De esta forma, trataremos de asegurar que el "peor caso" no ocurra.

Elegir el pibote:

Manera incorrecta:

La opción "más popular" es usar el primer elemento. Esta opción es aceptable, si el contenido de la lista es aleatorio, pero en el caso de que la lista este ordenada en orden reverso, se convierte en un caso extremo. Otra forma fácil de elegir el pibote es tomar el mayor de los dos primeros elementos de la lista. Esta selección tiene los mismos problemas.

Opción "Segura":

Elegir el elemento medio es una opción razonable (i.e. el elemento en la posición $(low+high)/2$). Si la lista tiene algun tipo de ordenamiento, este pivote es la elección "perfecta". Sin embargo generalmente este no es el caso.

Partición de la "Mediana-de-tres":*

Elejir el elemneto medio como pivote, nos asegura una reducción solo en los casos donde las listas ya tienen orden. En casos reales elección es (tan) arbitraria como elegir el primer elemento, y solo nos asgura que podra ordenar listas ordenadas "más rápido". El particionamiento de **mediana-de-tres** es un intento de elejir un pivote "promedio". El este algoritmo, calcularemos la **mediana** del primer, medio, y ultimo elemento.

La *mediana* de de un grupo N números es menor $\lceil N/2 \rceil$ -esimo. La media nos asgura una división pareja de los elementos. Lamentablemente, calcular la media de una lista consume un tiempo considerable. Trataremos de estimar una '*mediana represenativa*'. Es decir, buscaremos un conjunto de numeros (muestra) y calcularemos la mediana . Cuanto mayor la muestra, el resultado es mas preciso. Sin embargo añadira mas tiempo a nuestro algoritmo. Entonces? La "forma más fácil" de elegir el pivote, sera calculando la mediana del primer, medio, y ultimo elemento.

Ejemplo: Dada la lista $[8, 1, 4, 9, 6, 3, 5, 2, 7, 0]$ el primer elemento es 8, el último es 0, y el elemento central es 6, entoces eligiremos el número 6 como pivote.

0	1	4	9	6	3	5	2	7	8
---	---	---	---	---	---	---	---	---	---

Una forma sencilla es ordenar estos tres elementos en la lista (luego tomar el elemento medio).

0	1	4	9	7	3	5	2	6	8
---	---	---	---	---	---	---	---	---	---

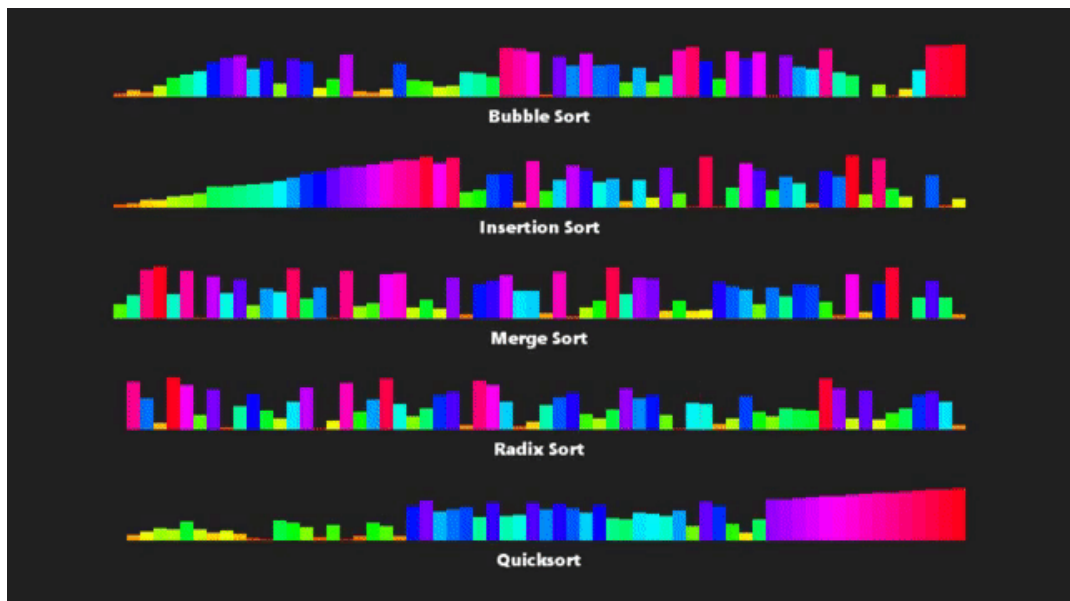
Ahora podemos notar que las siguientes condiciones se cumplen: (i) el elemento en la primer posición es menor (o igual) que el pivote, el elemento en la última posición es mayo (o igual) al pivote. De estas aserciones, podemos deducir lo siguiente:

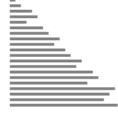
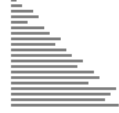
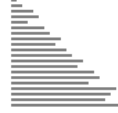
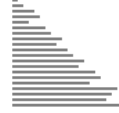
- No intercmbiar el pivote con el último elemento. En cambio, podemos insertarlo en el penultimo lugar.

8	1	4	9	6	3	5	2	7	0
---	---	---	---	---	---	---	---	---	---

- Podemos optimizar los indices: el contador i (izq. a der.) en $inicio+1$, y el contador j en $fin-2$.
- Cuando i recorre la lista buscando el mayor elemento, siempre terminara, ya que en el peor caso, será igual al pivote.
- Cuando j recorre la lista buscando el menor elemento, siempre terminara, ya que en el peor caso, será igual al primer elemento.

Comparación entre distintos algoritmos de ordenamiento.



								
 Random								
 Nearly Sorted								
 Reversed								
 Few Unique								